咕泡教育
—gupaoedu—

【Redis进阶直播课】

源码解读底层数据结构&过期策略&淘汰策略

主讲：天明

我们的愿景
推动每一次人才升级

我们的使命
让每个人的职业生涯不留遗憾

咕泡教育
—gupaoedu—

咕泡教育-天明

前银态优服架构师

前知名互联网学院Java讲师

咕泡教育架构师讲师

十几年JAVA从业经验。

拥有多年金融项目和大型互联网项目的架构设计经验，经历了消费金融公司从单体架构到大型分布式，微服务系统演变，支撑千万级客户的日均过亿的交易场景。

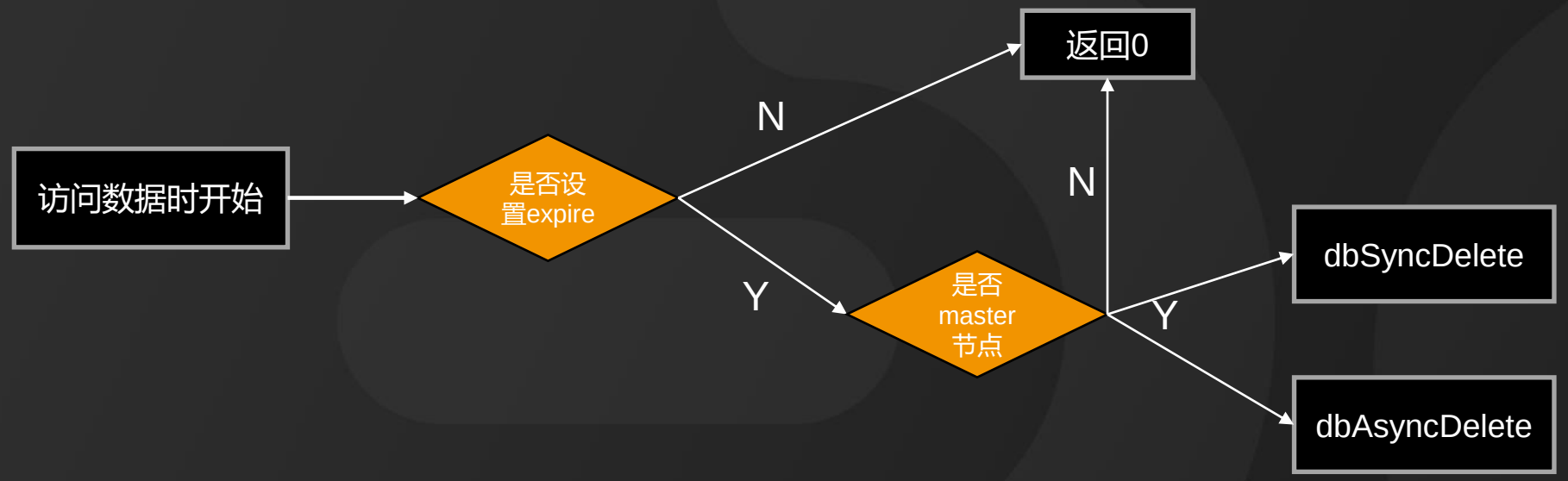
格言：明天到了，还看今朝

咕泡教育
—gupaoedu—

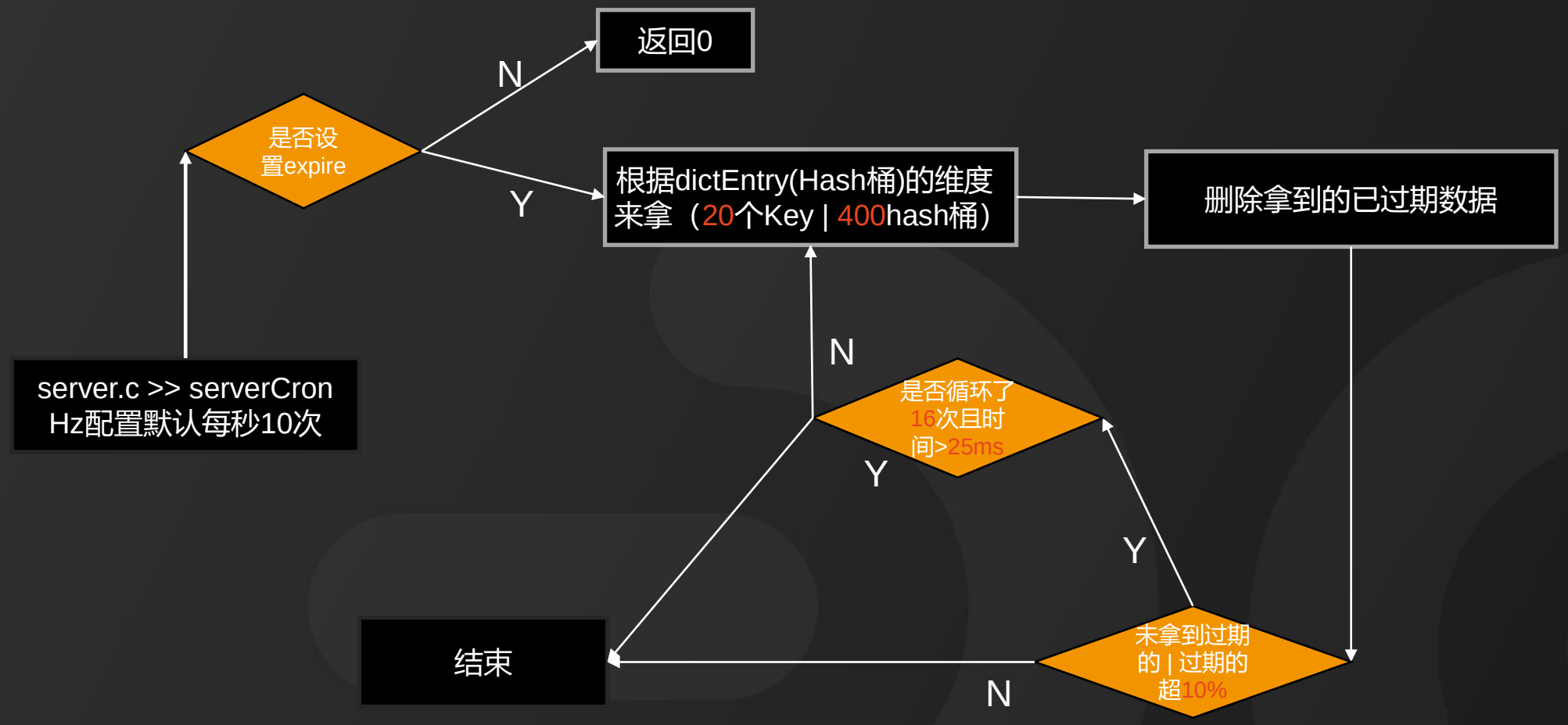
1 过期策略

惰性过期

```
db.c >> int expireIfNeeded(redisDb *db, robj *key) {
//如果没过期 返回0
if (!keyIsExpired(db,key))return 0;
//如果配置有masterhost主节点，说明这里是从节点，那么不操作增删改 不然就主从不一致了
 if (server.masterhost != NULL) return 1;
//判断 是否有在进行定期过期 两个方法 一个异步，一个非异步
int retval = server.lazyfree_lazy_expire ? dbAsyncDelete(db,key) : dbSyncDelete(db,key);
```



定期过期


server.c >> serverCron
Hz配置默认每秒10次
是否设置expire
N
返回0
Y
根据dictEntry(Hash桶)的维度来拿（20个Key | 400hash桶）
删除拿到的已过期数据
N
是否循环了16次且时间>25ms
Y
Y
未拿到过期的 | 过期的超10%
N
结束

咕泡教育
—gupaoedu—

2 淘汰策略

最简单的FIFO淘汰策略（Redis弃用此）

1，新增的元素添加到链表头部

2，移除链表尾部数据

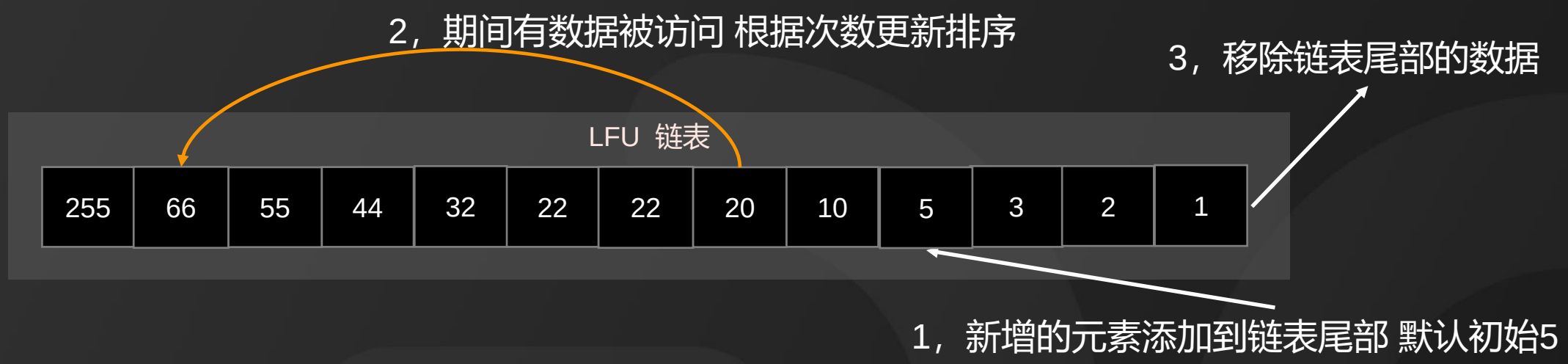
LRU算法(Least Recently Used)

2，期间有数据被访问到则移到头部

1，新增的元素添加到链表头部

3，移除链表尾部的数据

LFU算法(Least Frequently Used)



注意：访问次数相同则加入时间因素，为避免新增未被访问太容易淘汰给了初始5

TTL算法(Time to Live)

2，期间有数据被修改过期时间 根据时间更新排序

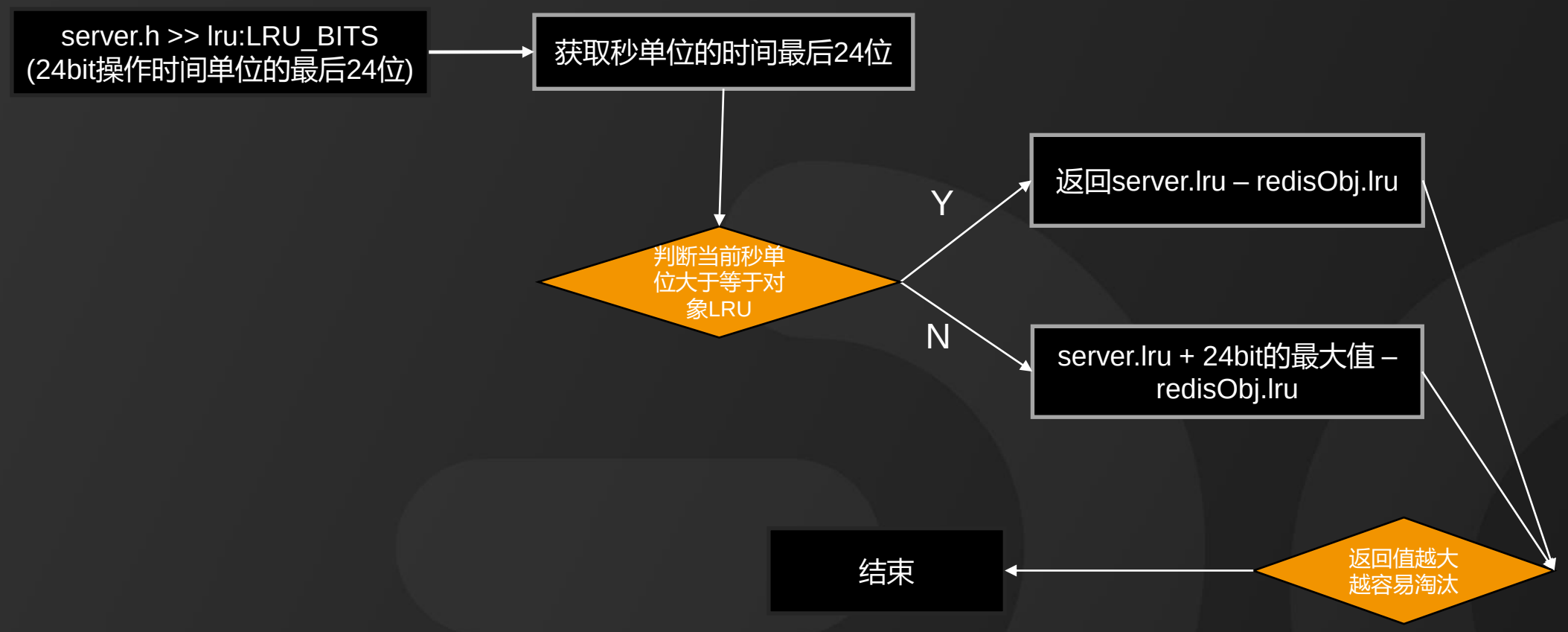
3，移除链表尾部的数据

剩余的过期时间

666	66	55	44	32	22	22	20	10	5	3	2	1

1，新增的元素 只针对设置了过期时间的key

LRU 源码解读



注意：allkeys-lru则省去判断是否设置了expire步骤

LFU 源码解读

server.h >> lru:LRU_BITS (16bit 操作时间 + 8bit的次数/频率)

Counter减的方法 LFUDecrAndReturn

- obj.lur字段右移16位得到16位时间
- lur字段与运算255得到8位counter值
- Lfu_decay_time默认一分钟减1次
- 非负数则用counter减衰减值

Counter加的方法 LFULogIncr

- 最大值255
- 减去基准值5，如果小于0设为0
- 如果counter小于5必然加一
- 否则根据baseval & lfu_log_factor 越大加的可能性越小

最后次数越少 越容易淘汰

结束

好评收集表

我们的愿景

打造一个技术人的 终身职业交流圈

咕泡教育
—gupaoedu—

欢迎大家加入 咕泡
和 30000+ 架构师共同成长

最后送大家一句话

天已明，夜未央!

咕泡教育
—gupaoedu—

谢谢观看

天明

我们的愿景
推动每一次人才升级

我们的使命
让每个人的职业生涯不留遗憾